Solution: Distribute Coins in Binary Tree

Problem Statement

You are given a `root` node of the binary tree, having `n` nodes, where each node contains the `node.val` number of coins. There are total `n` coins in the whole tree.

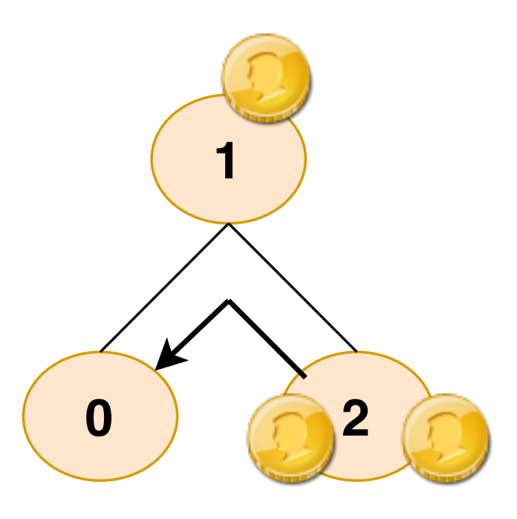
In one move, you can choose any `two adjacent` nodes and move a coin from one node to another. You can either move coins from `parent to child` or `child to parent`.

Return the `minimum` number of `moves` required to ensure every node has `exactly one` coin.

Examples

Example 1:

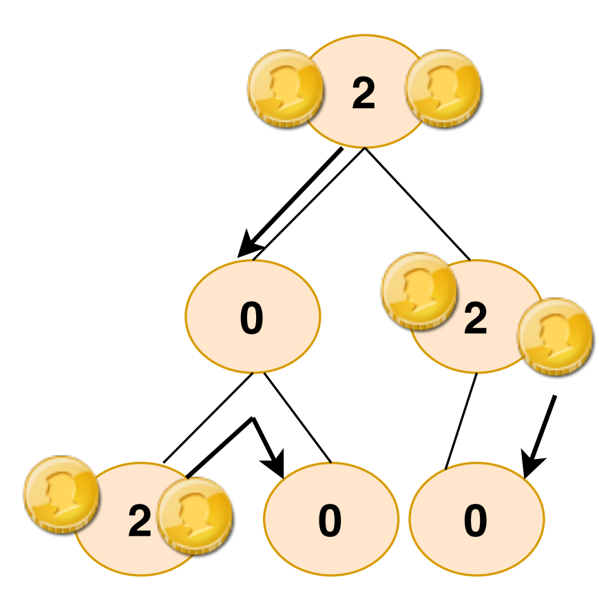
- **Input**: [1,0,2]



- **Expected Output**: 2
- **Justification**: We can move one coin from the right child to the root, and then from the root to the left child. This requires 2 moves to balance the coins.

Example 2:

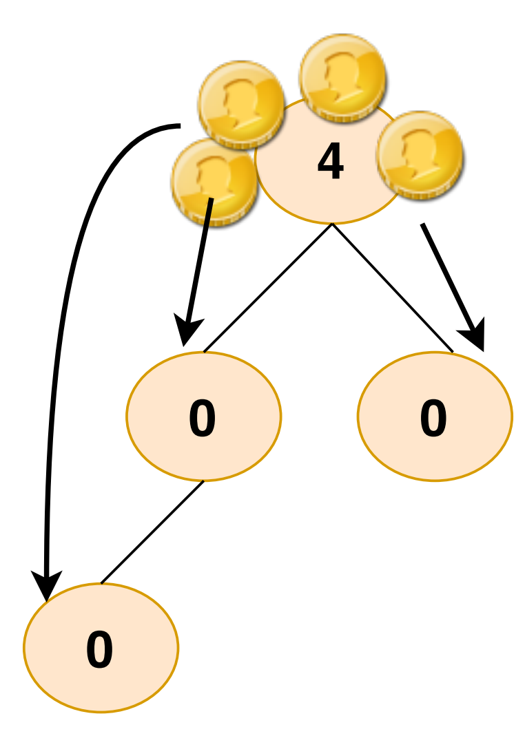
- **Input**: `[2,0,2,2,0,0]`



- **Expected Output**: 4
- **Justification**: Move one coin from the root node to its left node, which requires 1 move. After that, move one coin from root->left->left to root->left->right node, which requires 2 moves. Next, move one coin from root->right to root->right->left, which requires 1 move. Minimum number of moves required is `4`.

Example 3:

- **Input**: `[4,0,0,0]`



- **Expected Output**: 4
- **Justification**: Move two coins from the root to its left child, and then move one coin from this left child to its left child. This requires 3 moves. Move 1 coin to the right child, and this requires 1 move. So, total 4 moves are required.

Solution

To solve this problem, we'll adopt a post-order traversal approach, which ensures we deal with the children before the parent node. This method is effective because it allows us to calculate the surplus or deficit of coins at each node starting from the bottom of the tree. By doing this, we can determine how many coins need to be moved to or from each node to achieve the balance of one coin per node.

The key is to understand that the moves required to balance a child with its parent contribute to the total moves needed for the entire tree. This strategy is efficient as it minimizes the number of moves by directly addressing the imbalance at each node from the bottom up, ensuring that each move contributes to the final balanced state of the tree.

Step-by-step Algorithm

- Perform a post-order traversal of the binary tree. For each node, follow these steps:
  - Calculate the total coins in the left subtree (including the left child itself).
  - Calculate the total coins in the right subtree (including the right child itself).
  - Determine the number of moves required to balance the left and right children (absence of a coin is considered as a negative coin, indicating a need for a coin).
  - Update the total moves by adding the moves required for both children.
  - Calculate the number of surplus or deficit coins for the current node (coins in the current node plus coins from left and right children minus one for the current node).
  - Return the surplus or deficit of coins for the current node to its parent.
- The root node will not pass any surplus or deficit to a parent, as it has no parent. The total moves calculated during the traversal is the minimum number of moves required to balance the tree.

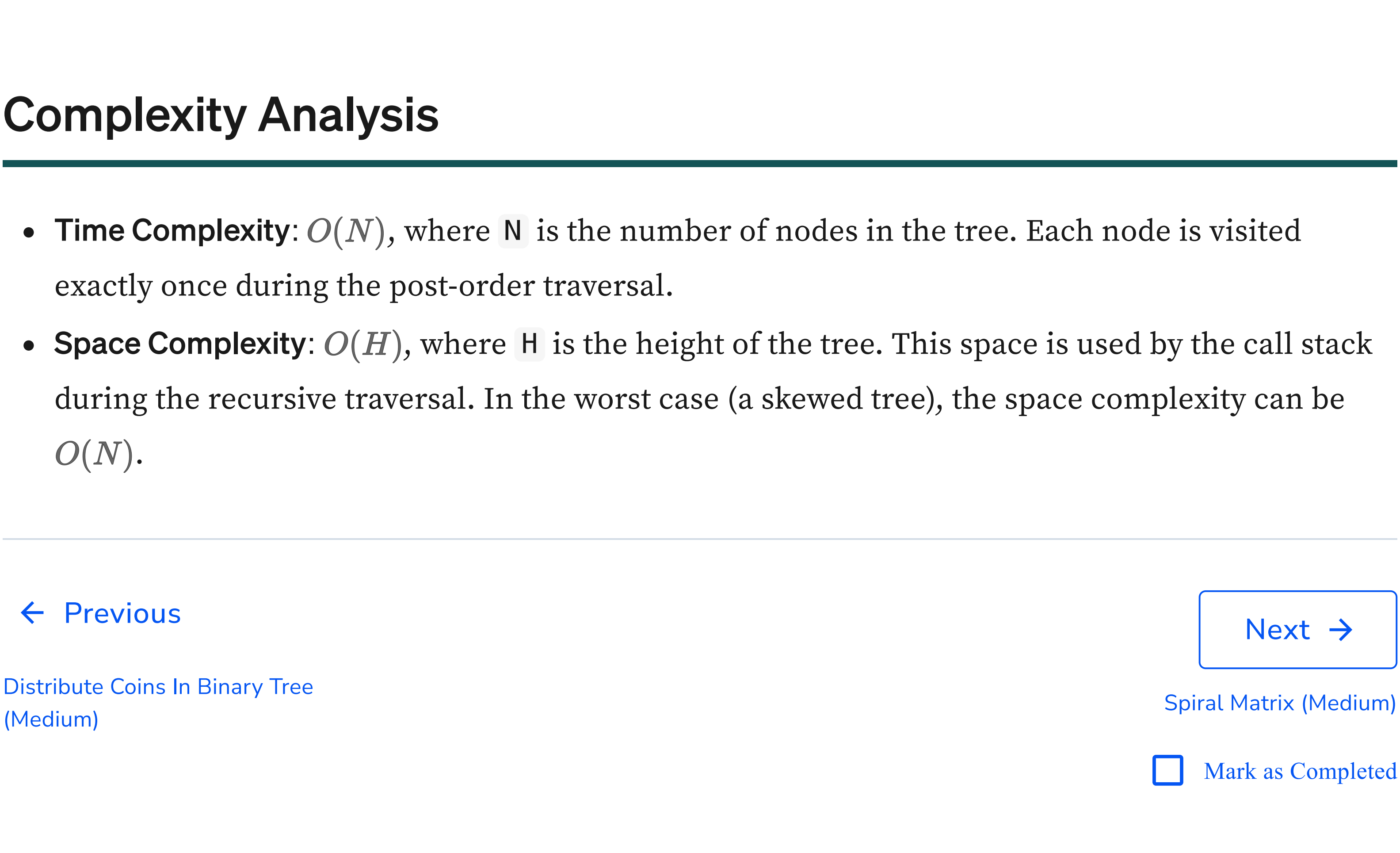
Algorithm Walkthrough

Let's walk through Example:

```
      2
     / \
    0   2
   / \  /
  2   0 0
```

1. **Start at the root** (**2**):
   - Begin the post-order traversal to balance coins in the tree.
   - `totalMoves` = 0 at this point, as no moves have been made.
2. **Move to the left child** (**0**) to balance this subtree first:
   - Continue traversal.
   - `totalMoves` still = 0.
3. **Visit the left child of the left child** (**2**):
   - This node has 2 coins and no children.
   - **Post-order action:** It has 1 excess coin (2 - 1 = 1). This excess will need to be moved to its parent (the 0 node).
   - **Moves required**: Moving this 1 excess coin contributes to 1 move.
4. **Visit the right child of the left child** (**0**):
   - This node has 0 coins and no children.
   - **Post-order action:** It has a deficit of 1 coin (0 - 1 = -1). This deficit will need to be fulfilled by moving a coin to it.
   - **Moves required**: Moving a coin to this node also contributes to 1 move.
5. **Back to the left child** (**0**):
   - `totalMoves` = `totalMoves` + `abs(leftExcess)` + `abs(rightExcess)` = 0 + `abs(1) + abs(−1)` = 2.
   - After balancing its children, it has a deficit of 1 coin because it gave 1 coin to each child but started with 0.
   - **Net balance:** It returns a deficit of 1 to its parent.
6. **Move to the right child** (**2**) **and then its left child** (**0**):
   - This node (the left child of the right child) has 0 coins.
   - **Post-order action**: It has a deficit of 1 coin (0 - 1 = -1). This deficit is returned to its parent (the right child).
   - **Moves required**: Moving a coin to this node contributes to 1 move.
7. **Back to the right child** (**2**):
   - `totalMoves` = `totalMoves` + `abs(leftExcess)` + `abs(rightExcess)` = 2 + `abs(−1) + abs(0)` = 3.
   - It has 0 excess coin after giving 1 coin to its left child.
   - **Net balance:** Returns 0 to the root.
8. **Back to the root** (**2**):
   - `totalMoves` = `totalMoves` + `abs(leftExcess)` + `abs(rightExcess)` = 3 + `abs(−1) + abs(0)` = 4.
   - The root now needs to balance the deficit from the left child with the excess coin.
   - **Moves required:** Moving 1 coin from the root to the left child directly requires 1 move.
   - After balancing, `totalMoves` = 4.

Code



```java
// class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;

//     TreeNode(int x) { val = x; }

//     TreeNode(int x, TreeNode left, TreeNode right) {
//         this.val = x;
//         this.left = left;
//         this.right = right;
//     }
// }

class Solution {
    int totalMoves; // Stores the total number of moves required

    public int balanceCoins(TreeNode root) {
        totalMoves = 0; // Initialize total moves to 0
        postOrderTraversal(root); // Begin post−order traversal to balance co[illegible]rti
        return totalMoves; // Return the total number of moves calculated
```

You must run your code first

Complexity Analysis

- **Time Complexity**: $O(N)$, where `N` is the number of nodes in the tree. Each node is visited exactly once during the post-order traversal.
- **Space Complexity**: $O(H)$, where `H` is the height of the tree. This space is used by the call stack during the recursive traversal. In the worst case (a skewed tree), the space complexity can be $O(N)$.

← Previous

Distribute Coins In Binary Tree (Medium)

Next →

Spiral Matrix (Medium)

☐ Mark as Completed